

EECS 470: Computer Architecture

Out-of-Order Processor Final Report

I . ABSTRACT

This report, prepared for EECS 470, presents the implementation of an N-way, R10K-based out-of-order processor. Our design incorporates advanced features such as early branch resolution, a gshare branch predictor, prefetching, and a blocking data cache with a 8.6ns clock period. This report details our design choices, performance analysis, and testing methodology to achieve this clock period.

II. INTRODUCTION

Over the past eight weeks, we have worked on building an N-way, R10K-based out-of-order computer processor with advanced features such as early branch resolution, a Gshare branch predictor, prefetching, and a blocking data cache. This report will provide a thorough walkthrough of this project, including a design overview, a high level diagram to visualize the modules and stages, explanations of individual modules and their performance, our testing methodologies, and potential future improvements.

III. DESIGN OVERVIEW

Our processor is N-way, EBR with a simpler memory system and prefetching. With EBR, we made a few decisions to make our branching logic simpler. We made sure that branch instructions were always the first and only branch instruction being dispatched per cycle. Below is our architectural diagram and each module is talked about in detail throughout the rest of this document.

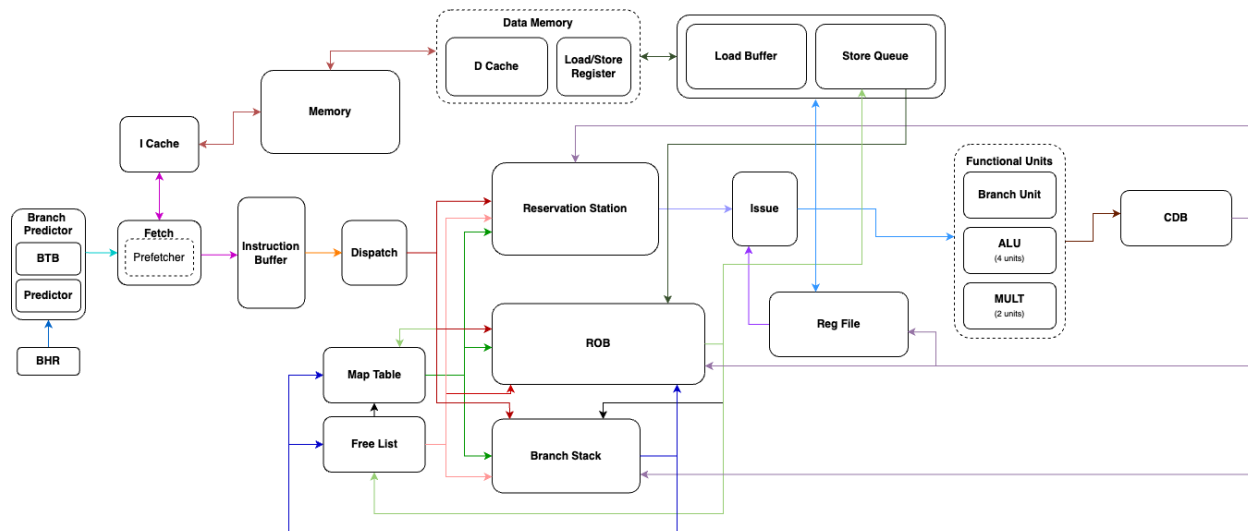


Fig. 1: High Level Diagram of our R10K Processor

To determine our N value for our processor, we analyzed the CPI compared to different possible N values. As seen in the graph below, we saw a significant decrease in CPI starting at N = 2. The marginal increase in the CPI decrease from 2 to 3 was greater than the CPI decrease from 3 to 4. We noticed that between N equal to 2 and 3, we were able to get much lower clock periods after synthesis than with 4 and 5. As a result, for our final N value, we chose 3.

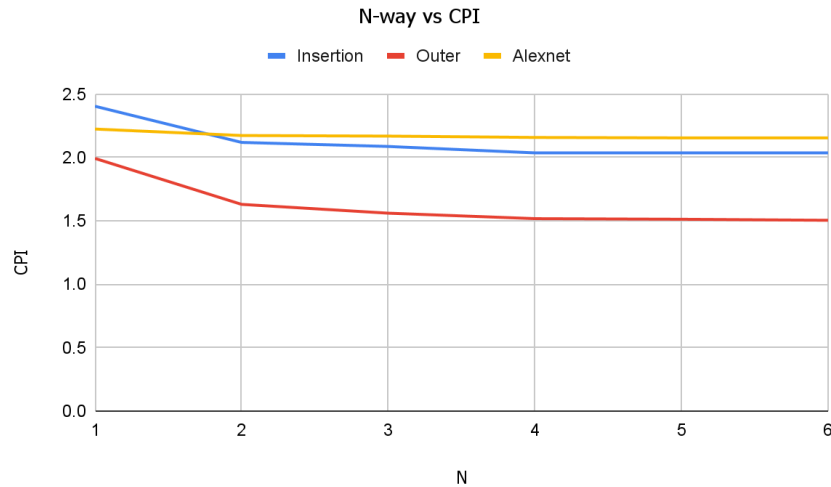


Fig. 2: N-way vs CPI

To determine the clock period, we started by finding the range of clock periods that we could synthesize at. We noticed that as our clock period decreased, our CPI increased due to the memory latency. To understand the tradeoff between our clock period and our CPI, we compared our clock period against the runtime to see which clock period would maximize the processor's performance as seen in the graph below. Our final clock period was 8.6.

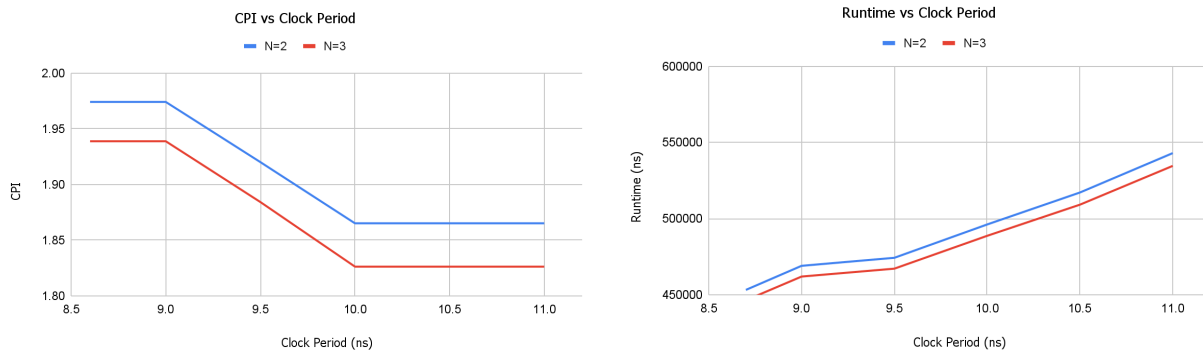


Fig. 3: Analysis to Balance Clock Period vs CPI

With N=3, we also looked at the average number of instructions being dispatched, issued, completed, and retired on the larger C files. We wanted to look at the average drop of instructions from each of our stages. The overall number of instructions dropped through each of our stages is very minimal, which shows that our throughput is consistent throughout the processor.

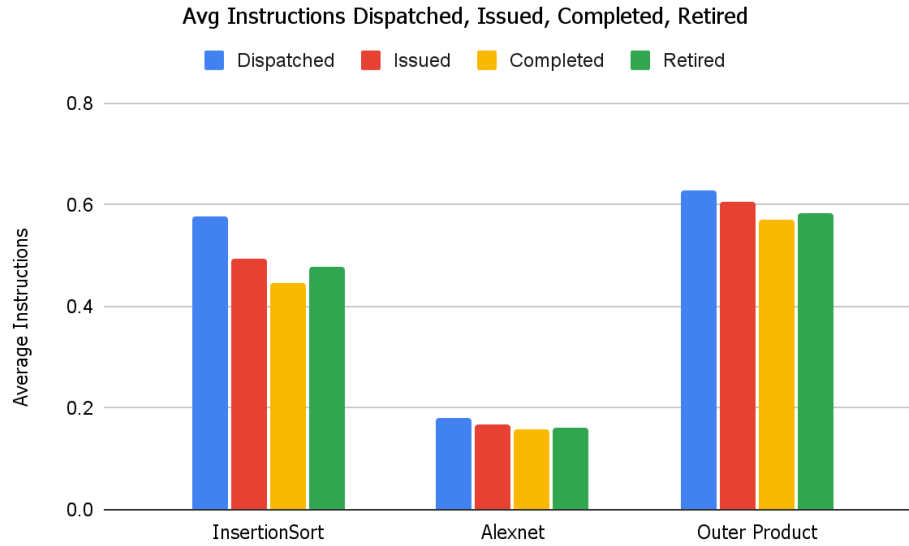


Fig. 4: Avg Instructions Dispatched, Issued, Completed, and Retired

IV. BASIC FEATURES & ANALYSIS

A. Dispatch

The dispatch module serves as an intersection point between the instruction buffer, the reservation station (RS) and reorder buffer (ROB). It transfers the instruction from the instruction buffer to the RS and ROB, according to conditions in the system. Dispatching in the current cycle stops when any of the following conditions are met:

- The limit of available spots in the ROB, RS, or number of valid instructions is met
- An illegal instruction is encountered
- A branch instruction is not the first instruction or the branch stack is full
- When the store queue (SQ) is full and the current instruction is a store

Dispatch produces several outputs; the RS and ROB receive the dispatched instructions to store accordingly, the freelist and instruction buffer receive the number of successfully dispatched instructions, and the SQ receives the dispatched store instructions. Each of these modules move forward with this data to act on the new set of instructions.

B. Reorder Buffer (ROB)

The ROB supports out-of-order execution while ensuring in-order retirement. It operates in a circular manner, with head and tail pointers; when new instructions are dispatched, the tail is incremented accordingly and the corresponding entries are written to the ROB. To retire instructions, the ROB iterates

over each instruction, stopping when it encounters an incomplete instruction. Retirement also halts if a memory stall occurs for store instructions or if there is a store operation in progress. Additionally, if an instruction has a halt condition and a store is in progress, retirement also stops. Based on the instructions successfully retired, the head is adjusted. The ROB will also pull any broadcasted values from the CDB, updating its entries accordingly. To support EBR, the ROB takes in a branch enable signal. On a mispredict, the ROB will restore the tail according to the saved branch tail, allowing it to recover its state.

One of the most prevalent features we chose to analyze for our processor is the size of the ROB, as this has a large impact on critical path and the number of out-of-order instructions that can be in-flight at any given time. As seen in the graph below, the ROB consistently performs better with a larger ROB size - though versions with 16 entries and 32 entries perform comparably across our testing suite, there are some key test cases involving many dependent instructions such as `outer_product.c`, which motivated the ultimate decision to use 32 entries. Additionally, we found that moving up from 16 entries to 32 didn't change our critical path significantly, which made the CPI increase worth the additional area.

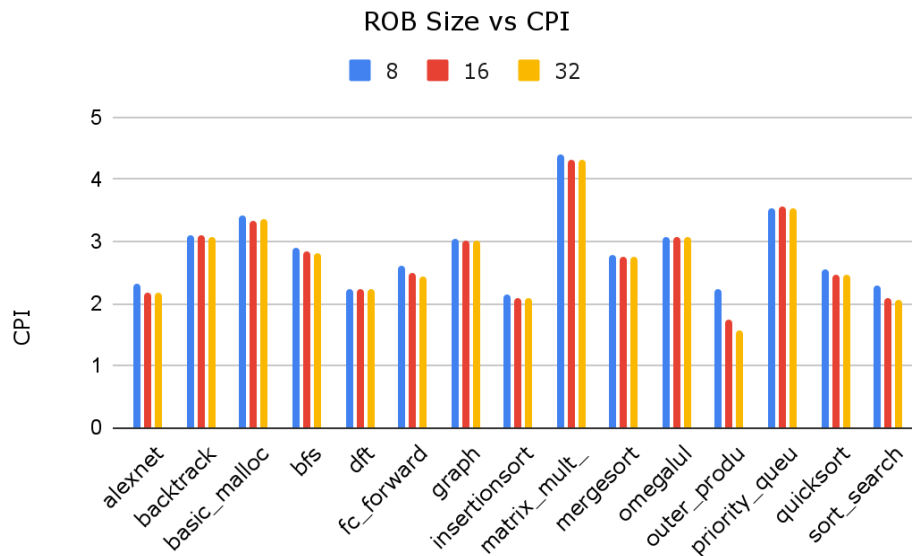


Fig. 5: Performance of Different ROB Sizes

C. Reservation Station (RS)

The RS is the heart and soul of our processor. It efficiently reads incoming instructions from the dispatch module and issues to our functional units. Every aspect of the RS is parallelized in hardware and we were able to avoid using any dependent loops. The key feature that enables this is our creation of an RS-specific priority selector, `rs-psel`, which combines two regular priority selectors combinationally. We instantiate a single instance of the `rs-psel` for each functional unit type and each `rs-psel` takes two request lines as input: a bit vector indicating each RS entry that has its dependencies resolved and is ready to issue, as well as a bit vector representing marking each functional unit that is currently busy. Using both these signals, the `rs-psel` outputs a single bus that associates each issued entry to the functional unit that it will be sent to. It also outputs a bit vector that marks each RS entry as issued and the total number of

instructions issued. We are confident that the rs-psel plays a major role in shortening our critical path and thus lowering clock period.

As part of our EBR implementation, we maintain a branch mask for every entry in our RS and take a branch id as input. During a squashing cycle, every entry that matches the squashing branch id will be deallocated from our RS. During a clearing cycle, every entry that matches the clearing branch id will remove that branch id from its corresponding mask.

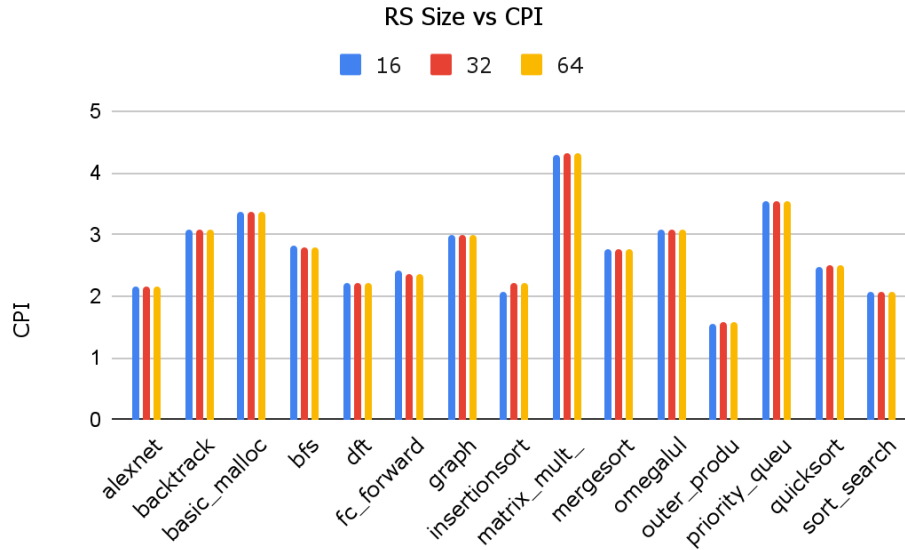


Fig. 6: Performance of Different RS Sizes

D. Freelist & Map Table

The freelist manages the allocation and return of physical registers in a circular manner, allowing our processor to handle register renaming and enabling out-of-order execution. When instructions retire (indicated by the number of instructions retired), the physical registers they used are returned to the freelist. These are added back in at the tail pointer. When instructions are dispatched, the freelist provides physical registers from the head pointer, dictated by the number of dispatched instructions. Each cycle processes writeback requests before read requests, allowing newly freed physical registers to be returned to use on the same cycle.

The free list also supports EBR by taking in two signals; a branch enable wire and a head pointer from the proper checkpoint, which is detailed later in the Branch Stack section. If a branch is mispredicted, the freelist will restore this saved head, undoing any register allocations made after said branch was dispatched.

The map table works alongside the freelist to enable register renaming by maintaining mapping between architectural registers and physical registers. When a freelist provides new physical registers, the

map table updates its entries for the architectural registers to point to these new physical registers; they are also marked as not ready until the value is broadcasted.

On each cycle, entries are marked as ready using the data broadcasted to the CDB. Similar to the freelist, the map table supports EBR by restoring to a saved version of the map table, ensuring the incorrect mappings made after the misprediction are reverted.

E. Issue Register

Our processor is built with 4 ALU units, 2 MULT units, 1 LD unit, 1 STORE unit, and 1 BR unit. The issue module sends instructions to these functional units, managing the flow of data and preserving issued instructions that aren't ready to be read by functional units. It receives data and instruction packets specifying the type of operation. In turn, each FU receives packets of data with the RS packet type and the rs1 and rs2 value.

The issue register module is also responsible for reading proper values from the register file. Our register file is implemented using a very simple data array with internal forwarding added. This allows the CDB to write to the register file on the same cycle as the issue module reads.

F. Functional Units (FU)

As previously mentioned, for our processor we chose to use 4 ALU units, 2 MULT units, 1 LD unit, 1 STORE unit, and 1 BR unit.

- The ALU units have been stripped down from their p3 equivalents, as they no longer need to perform operand selection; at a high level, each ALU unit is a series of logical computation units and a mux that uses the opcode to pick the correct computation unit.
- The MULT unit has been modified to carry more information across the multiplier stages, as each stage needs to keep track of its respective instruction packet. The multiplier presented unique challenges as the only pipelined FU, and thus features some additional complex features like propagation of the stalling signal across stages and flushing stages properly on a branch clear.
- The BR unit is responsible for both determining the correctness of the branch prediction and outputting the appropriate branch task, as well as calculating the proper target address for instructions that require it. To achieve this, it uses a series of comparator units alongside a simplified adder configured specifically for address calculation.

The FUs are not responsible for fetching their input values, instead receiving this data from the issue register. Each FU outputs to the CDB and follows a standard output structure, with LD and STORE having additional points of connection with the data memory system and BR having extra connections with the branch stack. Additionally, each unit is responsible for preserving information that the CDB doesn't have the capacity for; to this end, we've implemented a handshake between the CDB and FUs. When the CDB is at capacity, any additional ready FUs receive a stall signal, which tells the FU to maintain its current output; this means there are cases in which a stalled FU wouldn't be able to accept any new inputs.

As mentioned above, we implemented a ready-accept handshake with the CDB. The CDB accepts a single, unified ready signal that accumulates data ready signals from every FU. We maintain a single bit vector that is as long as the total number of FUs we add across every FU type and individual FUs correspond to a single bit on this vector. As part of our EBR implementation, we allocated the most significant bit of this vector to our branch unit. Since the CDB uses a priority selector to determine which

FUs will broadcast, setting the branch unit as the most significant bit ensures our branch unit will never stall.

With the exception of the ALU and BR unit, all our functional units are fully-pipelined. A major bug we encountered was erroneously issuing to a unit that would be stalled. Since we do not implement Early Tag Broadcast, we compute the CDB stalling signal after instructions are issued from the RS. We noticed a bug where our RS would issue to a FU that was not stalling on the current (issuing) cycle since the CDB was not marking that FU as stalling on this cycle. That instruction would then be written into our issue stage before being read into the FU on the next cycle. However, if the CDB decided to stall that same FU on the next cycle, that FU would stop accepting instructions from the issue register and, since the issue register is overwritten on every cycle, that specific instruction would end up getting dropped. To solve this issue we augmented our stalling signal to mark appropriate functional units as “busy” if there is a chance that the CDB will stall that unit on the upcoming cycle.

G. Common Data Bus (CDB) & Regfile

The CDB module manages broadcasting results from the functional units to components like the ROB and RS. When an FU completes execution, it raises a request through an FU done signal, which represents each of the FUs one to one. The CDB uses a priority selector to arbitrate these requests, allowing up to N FUs access to the CDB. The outputted stall signal indicates which FUs have not been selected and prompts them to hold onto their request till the next cycle. This signal prevents the FUs from overwriting their output or requesting data for a next execution prematurely.

Using the grant bus set by the psel, the CDB updates its entries with the register, physical register, computed value, and its validity. These entries, along with the stall signal, are then outputted.

The regfile module updates register values based on the broadcasts from the CDB. A write to the regfile occurs when the write enable signal is high and the target register is non zero. Reads are requested by the issue stage, indexing into the regfile’s data. The regfile supports simultaneous writes and reads are allowed, with write operations taking precedence in a cycle, ensuring the most updated data for any subsequent reads.

During our very first synthesis run, we attempted to synthesize at a 10ns clock period. However, we were unable to make slack as, at this point in the project, our CDB was fully combinational. Luckily, the synthesizer clearly identified the CDB as our critical path and we were able to quickly replace our combinational CDB implementation with a fully sequential implementation in a single day. The current, sequential CDB module enables us to achieve our low clock period.

V. ADDITIONAL FEATURES & ANALYSIS

A. ICache with Prefetching

Our Icache is configured to prefetch instructions from memory when the memory port isn’t in use by the data memory system; because our memory system has a single read port, we are able to fetch two

instructions from memory at a time to store in the cache. We have 15 MSHR entries, corresponding to the 15 possible tags that can be returned by the memory system, so we can have at most 15 in-flight prefetch requests. The Icache returns the address of the next sequential address that isn't a cache hit, which is the address we use to retrieve instructions from memory. The Icache will only fetch up to a certain distance; in order to optimize our processor, we analyzed this parameter across several testing scenarios, which can be seen below. Additionally, when prefetching instructions, we reference the branch predictor to see if any of these PCs correspond to a branch; if they do, then any subsequent prefetched instructions come from the target given by the branch predictor as opposed to the next sequential PC.

We chose a prefetch distance of 4 because there appeared to be minimal effect on CPI while maintaining a balance between memory use and latency. In general, increasing prefetch distance lowers the CPI because our out-of-order core is an instruction hungry processor, and bottlenecked by the number of instructions we can feed it. Prefetching allows our processor to look ahead in the instruction stream and maintain a larger instruction window while handling icache miss latencies. A prefetch distance of 4 (8 instructions) proved to be a balance between minimizing icache miss latencies and minimizing branch misprediction (MSHR + icache clearing) latencies. Prefetching 4 blocks enables us to also minimize the number of unused instructions that would have otherwise been prefetched.

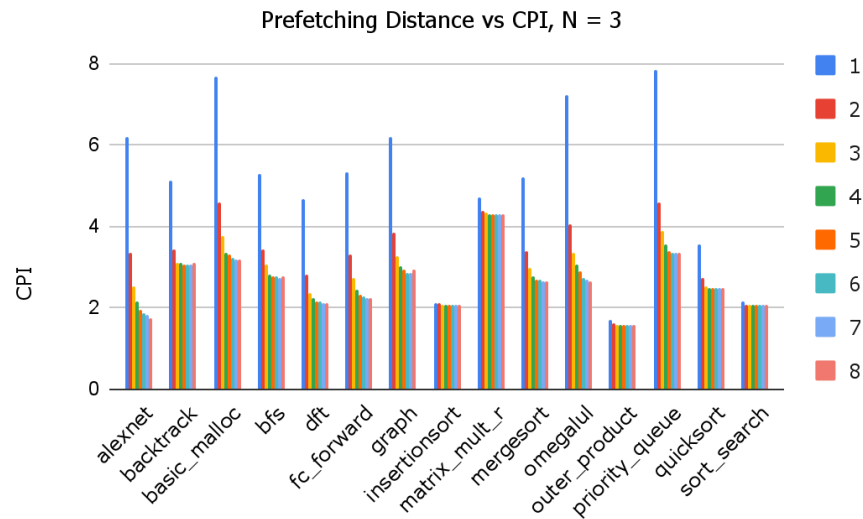


Fig. 7: Performance of Different Prefetching Distance

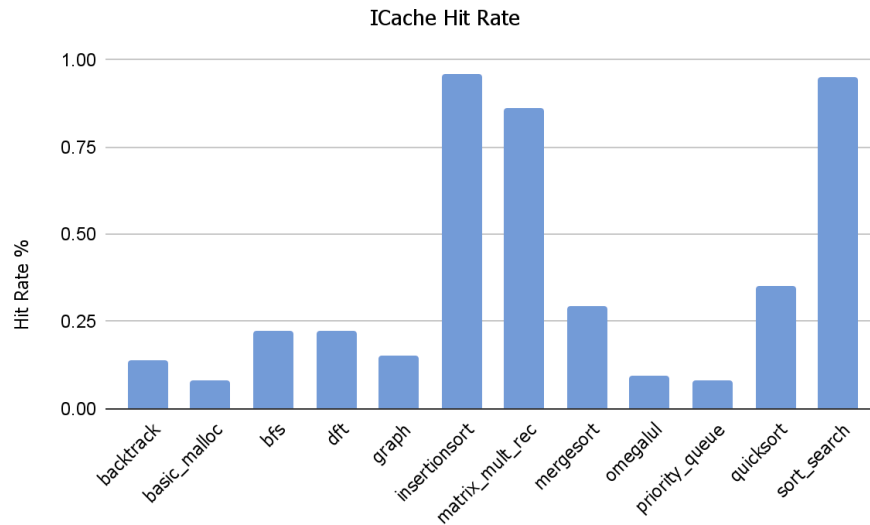


Fig. 8: ICache Hit Rate

B. Gshare Branch Predictor:

For branch prediction, we settled on implementing a GShare predictor. We made this decision based on its ease of implementation and its reported results in testing. This predictor allowed us to substantially improve our prediction rate from our previous “not taken” prediction. We implemented this predictor using a Branch Target Buffer to predict if an instruction is a branch. With this implementation, we would always predict “not taken” on the first occurrence of a branch, but will update the Branch Target Buffer once the branch is resolved and will know its a branch on the next occurrence. This also allows us to predict the target address of the branch instruction, squashing if we are incorrect in our prediction.

The GShare prediction model also utilizes a global Branch History Register, which gets updated whenever a branch leaves the branch functional unit. To support speculative branch prediction, we decided to preserve BHR state on the branch stack and restore on a squash, so incorrect speculative data doesn’t impact future predictions. The size of the BHR dictates how quickly the branch predictor warms up, so choosing a BHR size was critical for the accuracy of the predictor; after analyzing prediction rates across the different C programs, we found that 4 bits was a good compromise between accuracy and warm up time for the programs we test on. This smaller BHR size can be attributed to the fact that many of the testing files are relatively small, and that other elements of our predictor also take time to warm up. The first occurrence of a branch will always be a miss, as the BTB won’t identify the branch, and each 2 bit saturating counter takes time to train on the branches as well.

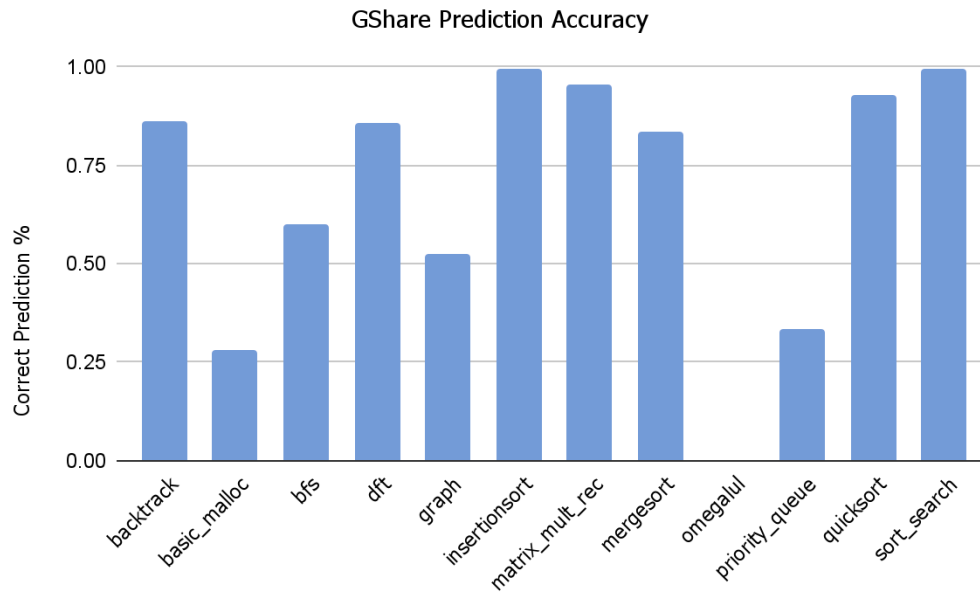


Fig. 9: GShare Prediction Accuracy

C. Branch Stack

The branch stack module enables our processor to handle early branch resolution (EBR). When a valid unconditional or conditional branch is sent in through dispatched instructions, always as the first instruction in the set, a checkpoint is created. This checkpoint stores a `b_mask`, `b_id`, map table, freelist head, ROB tail, and SQ tail. Additionally, if a branch is unconditional and writes to a destination register other than register 0, the checkpoint's map table is updated to an allocated physical register index that is marked as not ready.

The branch stack supports two distinct tasks, SQUASH and CLEAR. On a SQUASH, the branch stack outputs the checkpoint corresponding to the removing `b_id`. This entry is then cleared and marked as free, and the next `b_mask` is updated according to the remaining `b_ids` in entries.

On a CLEAR, the next `b_mask` is updated if dependent on the entry being cleared. The entry itself is cleared and marked as free. The `b_masks` of the remaining entries will no longer depend on the cleared entry's `b_id`.

Finally, branch stack updates all the checkpoint map tables based on what is broadcasted as complete in the CDB. By doing this, we ensure that upon restoring a checkpoint map table, it accurately reflects instructions that have been completed.

D. Store Queue (SQ)

We implemented a very simple store queue to maintain store instruction ordering and initiate memory store transactions. The store queue is implemented as a FIFO and is directly wired into the dispatch module as incoming store instructions are immediately allocated an entry in the store queue. This introduces another structural hazard as we must stall in dispatch if a store instruction cannot be allocated an entry in the SQ. However, as the table above shows, there is no noticeable impact on performance between the smallest SQ size and the largest SQ size.

Store instructions follow a slightly different lifecycle compared to other instructions as they do not require writing to the register file. This allows us to connect the SQ directly to the ROB and bypass the CDB entirely. Store instructions are marked as complete in the ROB as soon as they are issued from the RS. This allows the ROB to immediately retire store instructions and tell the SQ to initiate a store transaction. The ROB will only retire a single store transaction at time and will not retire a store instruction if the data memory signal is currently stalled.

Once the ROB retires a store instruction, the SQ immediately pops the current head of the SQ and sends the corresponding store transaction to the data memory system. At this point, even if the store transaction is currently processing in the data memory system, the actual instruction has retired and has left the pipeline entirely.

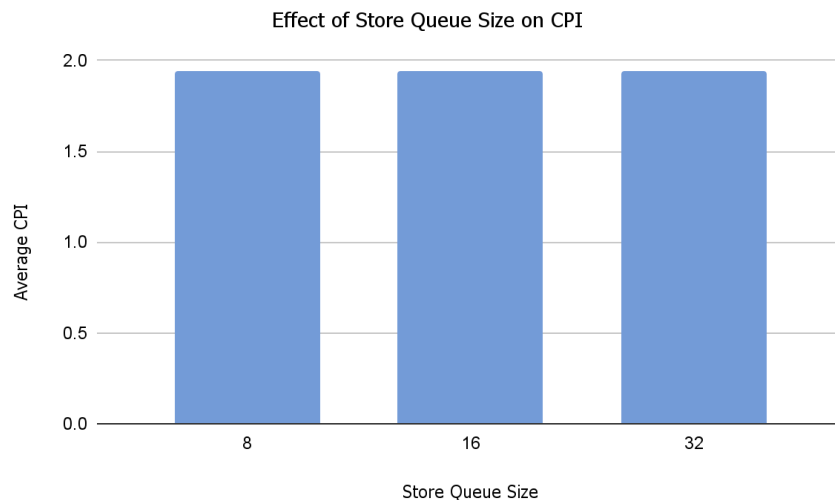


Fig. 10: Effect of Store Queue on CPI

E. Load Unit

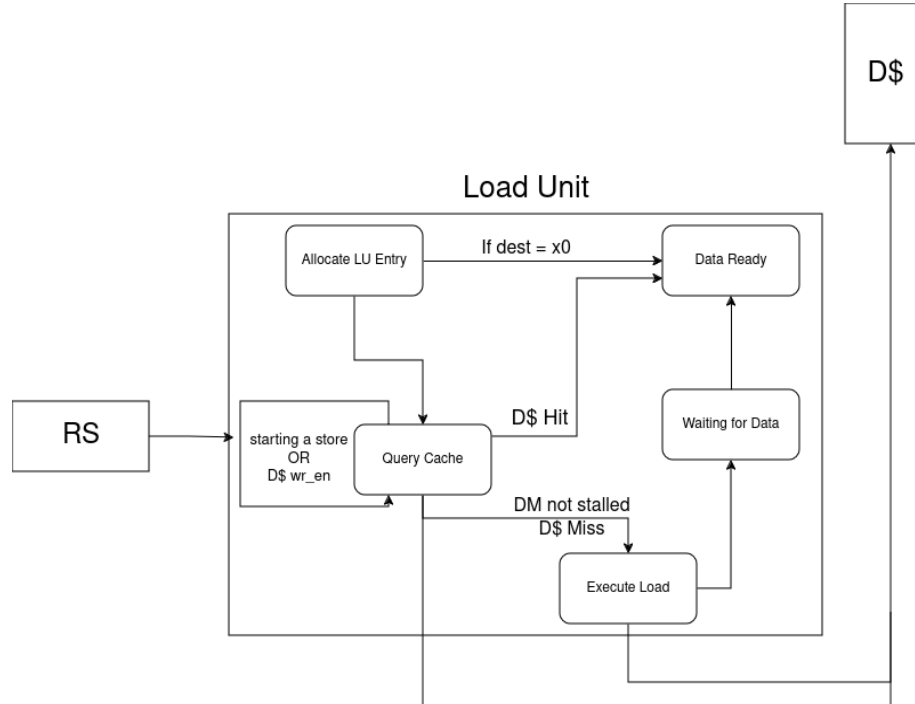


Fig. 11: Load Functional Unit State Diagram

Our load unit is implemented using a simple state machine. We enforce that the RS only issues 1 load instruction to the load unit at a time and stalls if the load unit buffer is full. Once a load instruction is issued, it is allocated an entry in the load buffer and marked as “ready to query”. The next cycle, if it is available, the DCache is queried. If the query is returned as a hit, that instruction is marked as “data ready”. Otherwise the instruction is marked as “ready to issue”. The next cycle, if the data memory system is not stalled and the instruction is chosen to be executed (via *psel*), that instruction is sent to the data memory system. At this point, the instruction is marked as “waiting for data”. Once the data memory system returns a memory block to the load unit, the load unit will query all entries in the load buffer and, if the memory tags match, will mark each corresponding entry as “data ready”.

Even though we chose to implement a blocking data memory system, we still decided to model our Load Functional Unit as if we chose to implement a non-blocking data memory system. We recognized that having a blocking data memory system does not necessarily force us to have a load unit of size 1 as the data memory system transfers a whole memory block and data coalescing occurs in the load unit for both blocking and non-blocking designs. We therefore expanded our load unit buffer to and, on every complete transaction, coalesced the incoming data for every instruction that requested the same memory block.

Additionally, we chose to implement a feature that forces each incoming load instruction to query our DCache in an attempt to increase DCache hits. The main use case for this feature is when a previous store instruction is in the data memory system, no other memory instruction can execute until that store completely resolves. However, that store instruction only reads from the DCache when it is popped from

the SQ and only writes to the DCache when the memory data tag returns. Therefore, any load that is issued between those two cycles has free access to the DCache. Our load issuing logic ensures that such loads are not dependent on the in-flight store. The only edge case is when a load instruction wishes to query the cache when a store needs access to the DCache. In this case, we forgo querying the DCache and mark that load instruction as ready to be sent to the data memory system in a later cycle. This is not exactly the optimal solution, especially if we have to forgo a query when the query would have been a DCache hit, but is necessary in order to ensure load instructions are forever stuck in a “query DCache” state.

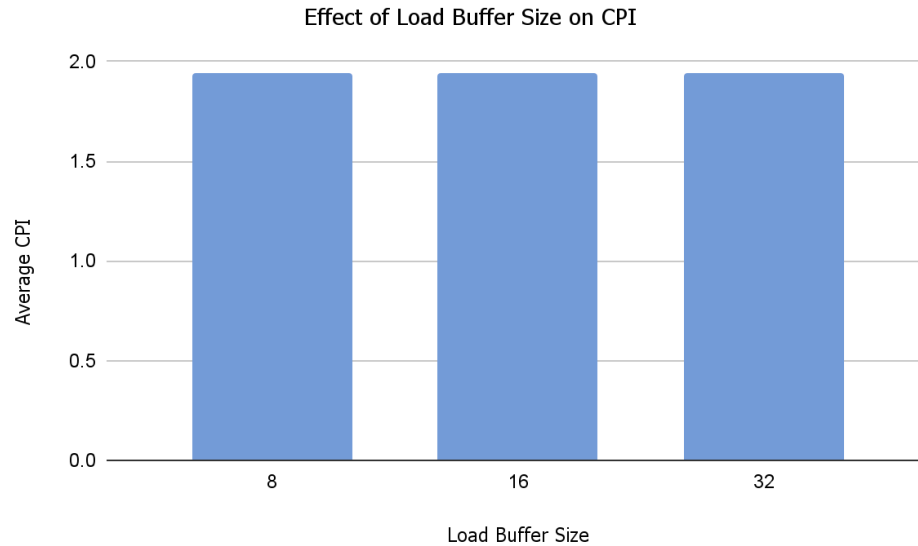
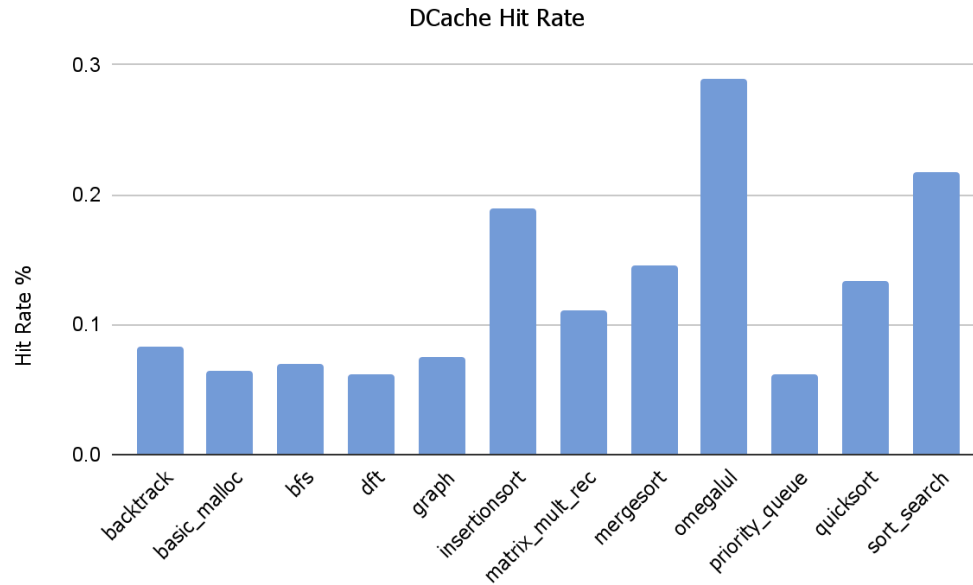


Fig 12: The Load Buffer Size has no noticeable Effect on Performance

F. DCache & Load/Store Register (Data Memory System)*Fig 13: DCache Hit Rate*

We implemented a blocking, write-alloc, write-through, direct-mapped DCache. Ultimately, we made the choice for our DCache system to be blocking because we believed that a non-blocking DCache would be too ambitious to implement in time for the final deadline. As such, our data memory system (DMS) is the biggest bottleneck in our pipeline. However, we designed our data memory system as if it were non-blocking on the off-chance we had time to implement non-blocking. Our data memory interface collects signals from our load unit and store queue and outputs 64-bit memory blocks back to the load unit as well as a stalling signal which will block any further loads or stores from entering the data memory system. Beyond the data memory system, or DMS, we maintain a 1-entry MSHR and the direct-mapped cache mentioned above.

For incoming loads, the DMS queries the DCache and, if a hit, immediately returns the full memory block on the same cycle. Otherwise, it allocates the MSHR entry and initiates a memory transaction, making sure to record the returned memory tag. The DMS continuously monitors the incoming data tag line and evicts the MSHR entry and corresponding data if the returned tag matches.

For incoming stores, the DMS queries the DCache and, if a hit, immediately dirties the cached data and initiates a memory store transaction. No MSHR entries are allocated in this case as memory stores are effectively instantaneous. Otherwise, an MSHR entry is allocated and a memory load translation is initiated. For stores, the MSHR is specially tagged as a store transaction. Once the load transaction is returned, the incoming data is dirtied and written to the DCache, the MSHR is deallocated, and a memory store transaction with the dirtied data is initiated.

Since we implemented a blocking and directly-mapped cache, we incurred a significant performance hit as we have a high number of cache misses and can only process one data memory transaction at a time. We tried alleviating this performance hit with our more complex load unit described above.

VI. TESTING

Our pipeline has had three forms of rigorous testing. With this testing, we were able to confirm correctness and feel confident in our processor through different types of programs for a range of N values (1 through 6).

A. Module Testbenches

Throughout the first and second milestones, we wrote individual testbenches for each module (ROB, RS, freelist, etc.) Through these, we tested for edge cases, basic functionality, and that each module performed individually. This led us to feel more confident in integration, and though we faced numerous bugs through integration, we believe our first level of testing contributed to an easier integration. We spend a significant amount of time building our module testbenches to be very resilient and, while we did stop running them closer to the end of the project, they enabled us to quickly iterate upon several module architectural changes while ensuring we maintained correctness with every change.

B. Unit Testbenches

Our second form of testing involves a unit test approach; for example, when we edited modules to account for a feature like EBR, small testbenches testing their functionality were written. With these small assembly testbenches, we could track which registers and instructions seemed to be running incorrectly and pinpointed errors in the modules.

C. Test Suite

The final and most significant form of testing has been running the test programs from Project 3. This is reflected across our performance graphs that are based on the 15 C programs. For every C and S program in the testing suite, we ran comparisons on our .wb file versus P3's .wb file, using a script we wrote. We've included a copy of this script in Appendix A below. The script runs our program for $N=1$ to $N=6$ and with every optimization flag to ensure that our processor was robust to handle multiple combinations of instructions. Both our initial fake fetch as well as our processor's fetch were utilized in testing with our script. We made this decision as fake fetch displayed a much higher throughput of instructions due to not needing to make requests to memory, and real fetch tested variability of number and order of incoming instructions, affording us further variety in our testing suite. We were able to confirm correct output through Milestones 2 and 3, as we integrated more modules into our pipeline.

VII. PROJECT MANAGEMENT

Our project management strategies fluctuated and improved heavily through the milestones. Admittedly, at the start, we did not have a clear plan nor division of work. Our administrative plan involved simply setting weekly check in meetings, lacking clear roles, assignments, and timeline expectations. This led us to have to regroup and reset goals as well as expectations for the latter half of the

project, as we rapidly fell behind our original timeline. By the final milestone, we had established a clear division of work and were much more synchronized as a team. We made the decision to drop a few specific features to have an achievable processor by the deadline, including the victim cache. Each member of the team was responsible for a different aspect of the modified project (data memory system, prefetching, etc.) To ensure we met our new goals, we had frequent check-ins with each other, online and in person, and held each other accountable. We consistently met every week throughout the duration of the project and in the later weeks, met multiple nights a week to work.

VIII. CONCLUSION & FUTURE STEPS

In conclusion, we developed an N-way, R10K-based out-of-order processor featuring early branch resolution, a gshare branch predictor, prefetching, and a blocking data cache with a 8.6ns clock period. This report details our design decisions, performance analysis, and testing methodology. Overall, this project provided a valuable learning experience and served as a great introduction to computer architecture.

If we were to continue iterating on this project, we would start by optimizing further. While our current performance analysis optimizes individual modules for the parameters that best boost their individual performance, future work would involve rigorous optimizations across each module. This method would cross reference parameters to understand what combination maximizes performance across all modules. We would also evaluate the impact of different types of branch predictors on the performance. Finally, we would aim to implement initially planned advanced features like the victim cache.